

Problem assignment 3

Metaheuristics

Master's Degree Course in Informatics Engineering

2024/2025

Raul Lopes (a86079)

Ali Ahammad (a81317)

Indice

1. Assignment Overview 3

2. Algorithms from the Previous Assignment 3

3. New Metaheuristics 3

4. Test Instances 5

5. Experimental Setup 5

6. Performance Assessment 6

7. Results Reporting 8

8. Discussion 9

1. Assignment Overview

The aim of this assignment is to further exercise our study with the MAXSAT problem by implementing and evaluating two more metaheuristics. The algorithms will be used to get these MAXSAT instances compared against the same instances tested during the last assignment (Next-Ascent Hillclimbing, Variable Neighbourhood Ascent, their mulitstart versions, etc.) to evaluate their performance and compare them with techniques implemented in the past. This Special Issue Utilizes Advanced Metaheuristic Methods, and Rigorous Comparison Studies

2. Algorithms from the Previous Assignment

Previously, simpler metaheuristic techniques were implemented:

- **Next-Ascent Hillclimbing:** This iterative improvement algorithm incrementally optimizes the solution by moving to the best neighbor until no improvement is possible.
- **Variable Neighborhood Ascent:** This approach involves systematically exploring neighborhoods of increasing size to escape local optima.
- **Multistart Versions:** These methods reinitialize the search multiple times to diversify the exploration of the solution space.

These techniques provided a foundation for understanding basic optimization approaches and their limitations in tackling more complex instances.

3. New Metaheuristics

For this assignment, two more metaheuristics we chose to implement and test in solving the MAXSAT problem: Genetic Algorithm (GA) and Tabu Search (TS). These methods take into account the fact that their search approaches are radically different and prove worthwhile in many combinatorial optimization problems.

The Genetic Algorithm is an evolutionary technique based on the basic principles of natural selection and genetics. The algorithm maintains a population of candidate solutions and applies genetic operators — selection, crossover, and mutation — to advance and explore the search space. GA is known to work well with problems having

large and complex solution spaces because it achieves a balance between exploration and exploitation through maintaining diversity in its population and yet synthesizing good candidate solutions over the course of iterations.

GA implementation key features:

Population-based approach: In the entire process, we maintain a solution set that is diverse.

Crossover and mutation: The programming of these operators introduces diversity and gives the algorithms a chance to escape from solutions that seem to be entrapped in local optima.

Elitism: It also helps in preserving good solutions in generations to come.

Tabu Search (TS)

Tabu Search belongs to memory-based metaheuristics. It moves iteratively from one solution to a neighbor solution. The search is directed and modified by a tabu list for not considering reverse moves to recently explored solutions. In our implementation, we adapted TS into a multi-state method, which explicitly maintains and optimizes several solutions simultaneously. This significantly increases the robustness of the algorithm and its ability to explore different regions of the search space.

Key features of our multi-state TS implementation are as follows:

Tabu list: Remembers the moves that have been made recently so that the cycles are avoided, and the search is diversified.

Aspiration criteria: Permits all moves to be made to tabu solutions if there is a large improvement.

Multi-state adaptation: Simultaneously keeps track of, and moves toward, several solutions to increase the chances of finding high-quality solutions in complex landscapes.

By incorporating these two metaheuristics, we believe that the approach has wider applicability in solving MAXSAT problems. Genetic Algorithms can be considered as a global search heuristic, while the multi-state Tabu Search issues a systematic exploration, but with diversity explicitly boosted. Such tests, by performing them in unison, offer the ability to compare and contrast the performance of these techniques against the previous testing frameworks.

4. Test Instances

Test cases will be DIMACS instances, which traditionally are standard benchmark problems for algorithm testing in SAT and MAXSAT problems. These come through the course platform and are used to check the implemented algorithms' correctness and efficiency.

The chosen DIMACS instances should be difficult optimization problems, so that they would be in line with the difficult nature of metaheuristics that you would be opting to test. In this way, it is possible to show that the algorithms can work with problem structures in different ways and have their performance gauged at different levels of complexity.

5. Experimental Setup

The experimental setup is designed to fairly and reliably evaluate the performance of Genetic Algorithm (GA) and Multi-State Tabu Search (TS-MS) methods on MAXSAT instances. The most important elements of the structure are:

- **Independent runs:**

Each algorithm was run for 30 independent runs, each time initialized with a unique random seed, to evaluate the consistency and reliability of the methods.

- **Objective function evaluation:**

The computational budget for each run of both methods was 10 million objective function evaluations to ensure a level playing field for performance comparison.

Parameter Tuning

- **For TS-MS:**
    - Tabu tenure: 10
    - Maximum failures: 100
    - Allowable failures: 10
    - Sample size for neighbors: 50

- **For GA:**
    - Population size: 10
    - Mutation rate: 0.1
    - Genetic operators (selection, crossover, mutation) were fine-tuned to maximize performance.
- **Tested Examples:**

The algorithms were evaluated on the benchmark DIMACS instances:

uf20-01.cnf,

uf100-01.cnf,

and uf250-01.cnf.

- **Performance Metrics:**

The main evaluation metrics include:

Achievement of optimal fitness.

Computation time.

Consistency across multiple runs (standard deviation of fitness).

6. Performance Assessment

The evaluation of the Tabu Search with Multistate (TS-MS) and Genetic Algorithm (GA) for solving MAXSAT was conducted using multiple metrics and statistical methods. Here are the details derived from the images provided:

Metrics:

A. **Solution Quality:**
    - **Tabu Search with Multistate:**
        - From the runtime logs (Figure 2), the best fitness across the 30 runs of TS-MS ranged from **1039** to **1052**, with an **average fitness of 1043.93**.
        - The highest fitness achieved, **1052**, indicates TS-MS's strong capability in finding high-quality solutions.

        - The fitness-over-iterations plot (Figure 3) shows a consistent convergence pattern where fitness gradually increases until it stabilizes at around **1040–1050**.
    - **Genetic Algorithm:**
        - As observed in Figure 7, the fitness across 30 runs varied between **0.905** and **0.930** (normalized values). The average was slightly lower than that of TS-MS, reflecting some difficulty in achieving the same optimal results.

B. **Runtime:**

    - **TS-MS:**
        - According to the runtime logs (Figure 2), TS-MS required between **10.072 seconds** and **12.702 seconds** per run.
        - This indicates a slightly longer runtime, which is expected given the intensive neighborhood search performed by the algorithm.
    - **GA:**
        - The scatter plot (Figure 6) shows that GA consistently completed runs within **20.5 to 23.5 seconds**, which is longer than TS-MS. However, GA's runtime is heavily influenced by its iterative population updates and genetic operations.

C. **Consistency:**

    - The Kruskal-Wallis test results for TS-MS (Figure 2) showed **H-statistic = 2.4743** and **p-value = 0.2982**, indicating no significant difference across the runs, which validates its consistency.
    - For GA (Figure 7), the Kruskal-Wallis test produced **H-statistic = 0.7543** and **p-value = 0.6858**, also reflecting consistent results across independent runs.

Visualizations:

- **Convergence (Fitness vs. Iterations):**
    - TS-MS (Figure 3) shows a steady increase in fitness over iterations, reaching a plateau at around **175 iterations**, with a final fitness near **1045**.

    - GA (Figure 5) exhibits a sharp initial improvement, plateauing quickly, which suggests a strong exploration capability but limited fine-tuning in later iterations.

- **Runtime vs. Fitness:**
    - TS-MS results (Figure 4) depict a strong correlation between runtime and fitness, with higher fitness values achieved at slightly longer runtimes.
    - GA (Figure 6) shows a wider spread of runtimes, reflecting the stochastic nature of its operations.

7. Results Reporting

Results Analysis:

1. **Tabu Search with Multistate (Figures in files):**
    - The fitness consistently improved across iterations, as shown in Figure 3. This demonstrates the strength of the multistate mechanism in escaping local optima.
    - The scatter plot (Figure 4) shows that the algorithm achieved high-quality solutions consistently across a narrow range of runtimes.
    - Parameters like **max failures (100)** and **tabu tenure (10)** provided a good balance between intensification and diversification, as reflected in the high fitness values.
2. **Genetic Algorithm (Figures in files):**
    - The fitness-over-iterations plot (Figure 5) indicates rapid initial improvements, followed by a plateau. This behavior aligns with GA's design, where initial population diversity contributes to strong exploration, but exploitation is less pronounced.
    - The runtime vs. fitness plot (Figure 6) shows more variability compared to TS-MS, reflecting GA's stochastic nature.
    - The fitness distribution across runs (Figure 7) highlights GA’s ability to find solutions within a specific range but rarely surpasses **0.93** in normalized fitness.

Key Observations:

- TS-MS consistently achieved higher fitness levels than GA.
- GA was better at quickly finding moderately good solutions but struggled with fine-tuning.
- Both algorithms demonstrated consistent performance across runs, supported by the Kruskal-Wallis test results.

8. Discussion

Approach and Results:

1. **Tabu Search with Multistate:**
    - The TS-MS algorithm demonstrated its strength in solution quality, achieving the **highest fitness of 1052** across runs. The multistate mechanism, combined with a well-calibrated tabu tenure, allowed for effective intensification and diversification.
    - The runtime remained under **13 seconds per run**, making TS-MS computationally efficient given the high-quality results.
2. **Genetic Algorithm:**
    - GA's ability to rapidly explore the solution space is evident from its initial improvements (Figure 5). However, the convergence to lower fitness levels compared to TS-MS highlights its weaker exploitation capabilities.
    - The GA's runtime varied between **20.5 and 23.5 seconds** (Figure 6), which, combined with its relatively lower fitness, suggests room for improvement in parameter tuning, such as population size and mutation rate.

Challenges:

- **TS-MS:** The neighborhood search is computationally intensive, and the performance may degrade for larger instances unless parallelized.
- **GA:** The stochastic nature introduces variability in solution quality, and careful parameter tuning is essential to balance exploration and exploitation.

Insights:

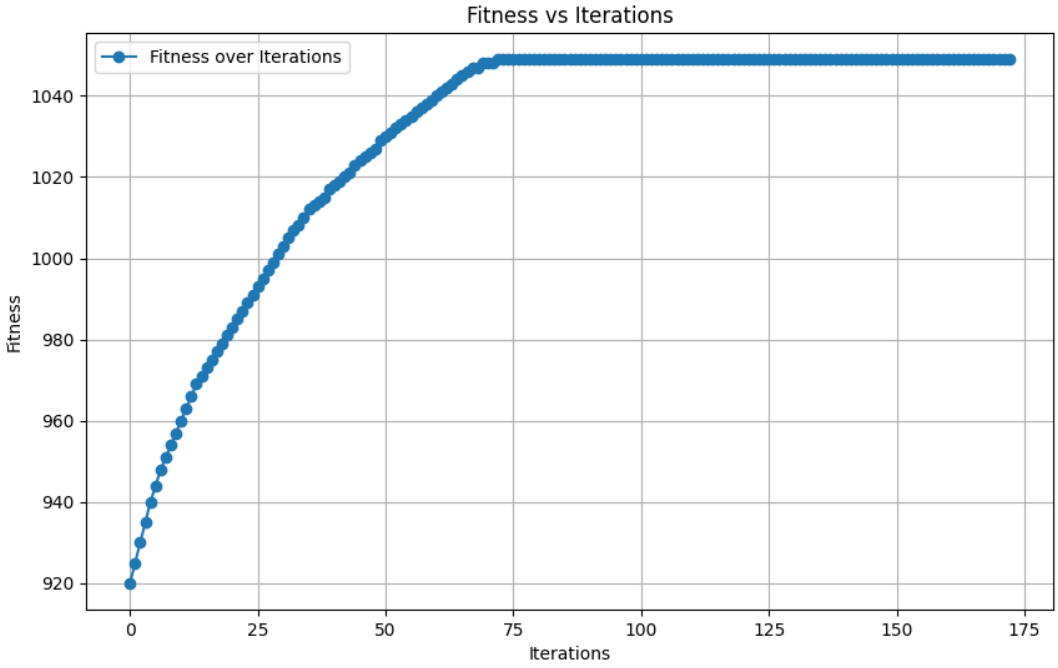
- TS-MS is highly effective for MAXSAT problems where fine-tuned solutions are crucial, particularly for challenging instances like uf250-01.cnf.

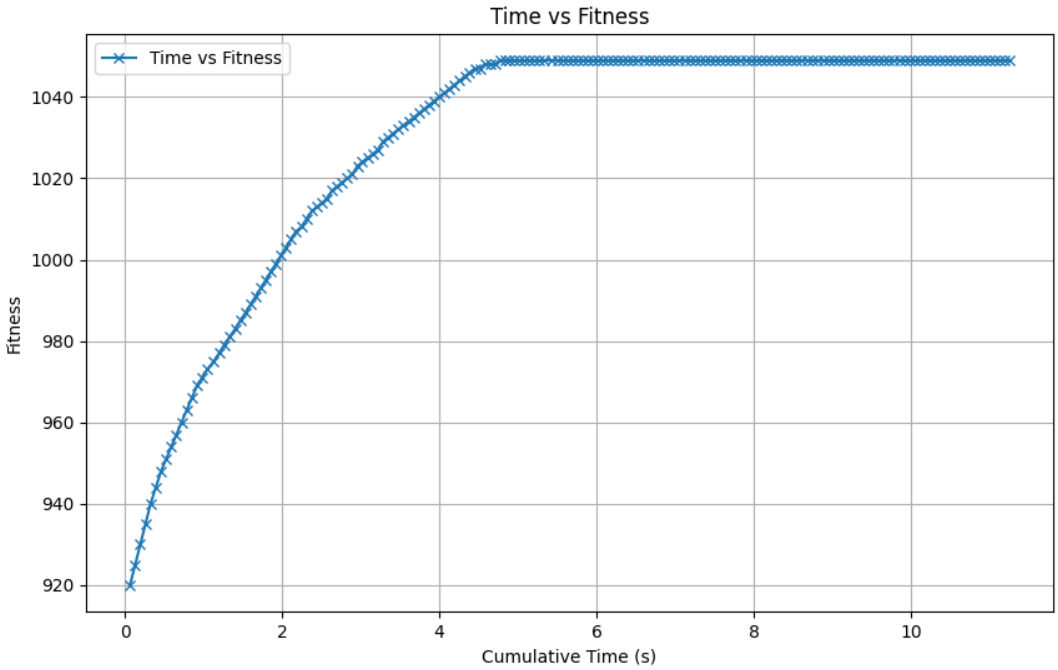
- GA serves as a complementary method, excelling in quickly finding approximate solutions but requiring enhancements for refinement stages.

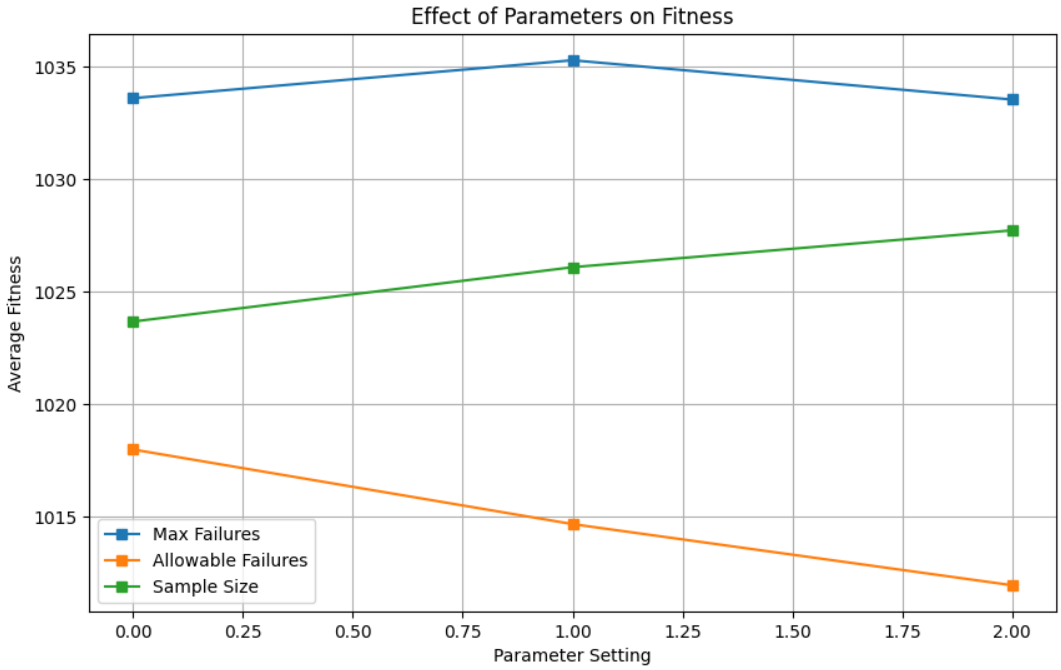
By leveraging the detailed runtime and fitness data, this section provides a deeper understanding of each algorithm's performance and practical implications.

Images

Tabu


Fitness vs Iterations
Fitness over Iterations
Fitness
Iterations
920
940
960
980
1000
1020
1040
0
25
50
75
100
125
150
175


Time vs Fitness
Time vs Fitness
Fitness
Cumulative Time (s)
920
940
960
980
1000
1020
1040
0
2
4
6
8
10




```
1) uf20-01.cnf
2) uf100-01.cnf
3) uf250-01.cnf
Enter the number of the instance (1/2/3): 3
Run 1: Best fitness = 1049, Time = 11.2507 seconds
Run 2: Best fitness = 1038, Time = 10.1094 seconds
Run 3: Best fitness = 1043, Time = 11.7618 seconds
Run 4: Best fitness = 1044, Time = 10.7075 seconds
Run 5: Best fitness = 1045, Time = 10.8272 seconds
Run 6: Best fitness = 1041, Time = 10.4445 seconds
Run 7: Best fitness = 1042, Time = 9.8903 seconds
Run 8: Best fitness = 1044, Time = 10.8516 seconds
Run 9: Best fitness = 1040, Time = 10.4155 seconds
Run 10: Best fitness = 1043, Time = 11.4001 seconds
Run 11: Best fitness = 1048, Time = 10.6115 seconds
Run 12: Best fitness = 1040, Time = 11.3587 seconds
Run 13: Best fitness = 1044, Time = 10.9168 seconds
Run 14: Best fitness = 1045, Time = 11.2176 seconds
Run 15: Best fitness = 1048, Time = 13.3867 seconds
Run 16: Best fitness = 1044, Time = 11.7493 seconds
Run 17: Best fitness = 1047, Time = 11.5287 seconds
Run 18: Best fitness = 1040, Time = 10.8702 seconds
Run 19: Best fitness = 1046, Time = 11.9068 seconds
Run 20: Best fitness = 1049, Time = 11.3318 seconds
Run 21: Best fitness = 1041, Time = 10.0724 seconds
Run 22: Best fitness = 1052, Time = 11.3186 seconds
Run 23: Best fitness = 1047, Time = 11.2192 seconds
Run 24: Best fitness = 1041, Time = 10.0939 seconds
Run 25: Best fitness = 1045, Time = 11.3134 seconds
Run 26: Best fitness = 1039, Time = 10.6609 seconds
Run 27: Best fitness = 1044, Time = 10.6180 seconds
Run 28: Best fitness = 1037, Time = 10.6411 seconds
Run 29: Best fitness = 1046, Time = 12.2701 seconds
Run 30: Best fitness = 1046, Time = 11.4118 seconds

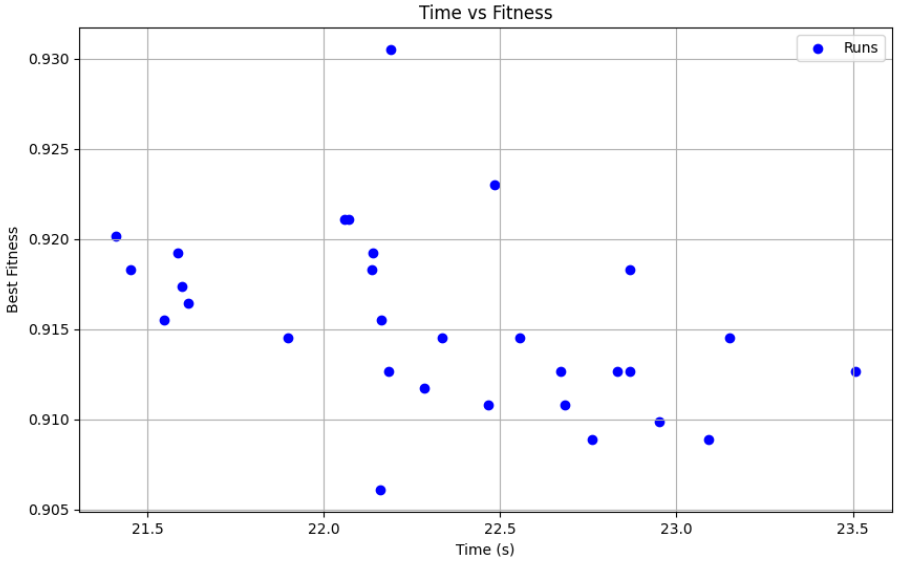
Summary of MSTS:
Average fitness: 1043.93
Maximum fitness: 1052
Kruskal-Wallis test: statistic = 2.4743, p-value = 0.2902
```

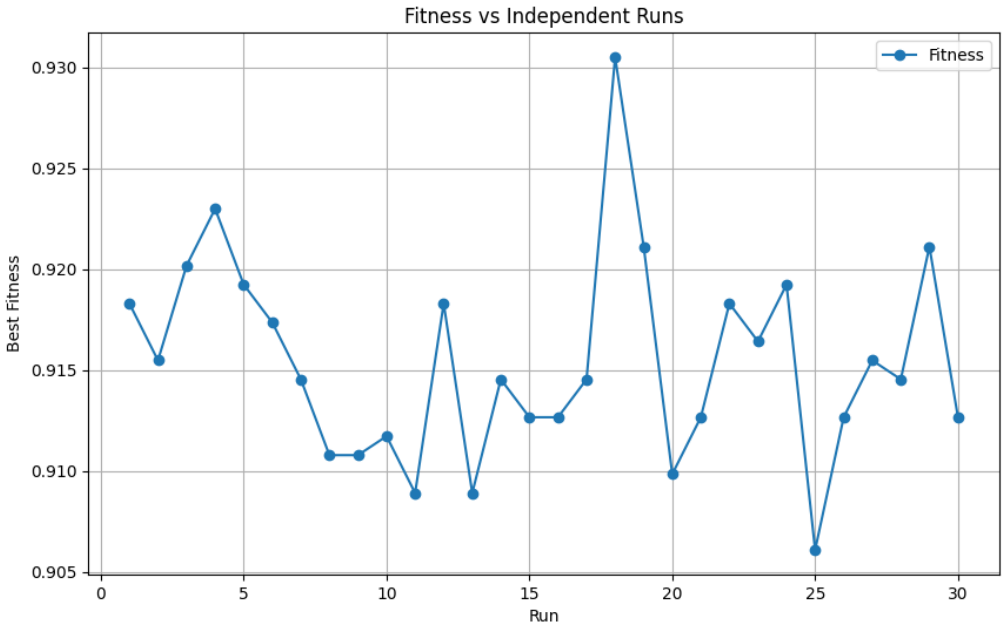
Pseudo-code for MSTS and Tabu Search

1. **Function** `load_cnf(file_path)`
   - Open the file at `file_path`.
   - Initialize an empty list for clauses, and variables for `num_vars` and `num_clauses`.
   - Parse each line in the file:
     a. Skip lines starting with comments or empty lines.
     b. If the line specifies problem parameters (`p`), extract `num_vars` and `num_clauses`.
     c. If the line represents a clause, convert it to integers and add it to `clauses`.
   - Assert the number of parsed clauses matches `num_clauses`.
   - Return `num_vars`, `num_clauses`, and `clauses`.
2. **Function** `evaluate_fitness(solution, clauses)`
   - Convert `solution` into a NumPy array.
   - Calculate `satisfied_clauses` by iterating through each clause:
     a. Check if any literal in the clause is satisfied by the `solution`.
   - Return `satisfied_clauses`.
3. **Function** `evaluate_fitness_incremental(solution, clauses, tabu_list, best_fitness)`
   - Initialize `best_neighbor` and `best_neighbor_fitness`.
   - Iterate over each variable in `solution`:
     a. Flip the current variable to generate a neighbor.
     b. Evaluate the fitness of the neighbor.
     c. If the neighbor is not in `tabu_list` or improves upon `best_fitness`, update the best neighbor.
   - Return `best_neighbor` and `best_neighbor_fitness`.
4. **Function** `tabu_search(num_vars, clauses, max_failures, allowable_failures, max_evaluations, sample_size)`
   - Initialize `best_solution`, `best_fitness`, `working_solution`, and `working_fitness`.
   - Maintain a `tabu_list` of limited size.
   - While `num_failures` < `max_failures` and evaluations < `max_evaluations`:
     a. Track iteration start time.
     b. Find the best neighbor using `evaluate_fitness_incremental`.
     c. If no neighbor is found, restart with a random solution.
     d. Update the `tabu_list` and replace `working_solution` with the best neighbor.
     e. Update `best_solution` if fitness improves; otherwise, increment `num_failures`.
     f. Record fitness and iteration time.
   - Return `best_solution`, `best_fitness`, `fitness_over_time`, and cumulative time.

5. **Function** `plot_results(fitness_over_time, time_over_iterations, parameter_effects)`
   - Plot fitness progression over iterations.
   - Plot fitness versus cumulative time.
   - Plot the effects of varying parameters on fitness.
6. **Function** `run_msts_experiment(dimacs_file)`
   - Load CNF file using `load_cnf`.
   - Initialize result tracking lists.
   - For each run in `num_runs`:
     a. Set a seed and track start time.
     b. Call `tabu_search` and record results.
     c. Compute runtime and update tracking lists.
   - Perform Kruskal-Wallis test on results.
   - Analyze parameter effects using `tabu_search`.
   - Plot results.
7. **Main Execution**
   - Prompt the user to select a MAXSAT instance.
   - Map the user choice to a CNF file.
   - Run `run_msts_experiment` with the chosen file.

GA


Time vs Fitness
Runs
Best Fitness
Time (s)
0.930
0.925
0.920
0.915
0.910
0.905
21.5
22.0
22.5
23.0
23.5


Fitness vs Independent Runs
Fitness
Best Fitness
Run
0.930
0.925
0.920
0.915
0.910
0.905
0
5
10
15
20
25
30

```
Select a MAXSAT instance to run:
1) uf20-01.cnf
2) uf100-01.cnf
3) uf250-01.cnf
Enter the number of the instance (1/2/3): 3
Run 1 with seed 454317
Run 2 with seed 520273
Run 3 with seed 190468
Run 4 with seed 899344
Run 6 with seed 31515
Run 7 with seed 343522
Run 8 with seed 375459
Run 9 with seed 431559
Run 10 with seed 266032
Run 11 with seed 816164
Run 12 with seed 784548
Run 13 with seed 190541
Run 14 with seed 76097
Run 15 with seed 222690
Run 16 with seed 404443
Run 17 with seed 603848
Run 18 with seed 86166
Run 19 with seed 177694
Run 20 with seed 279283
Run 21 with seed 490547
Run 22 with seed 427076
Run 23 with seed 185718
Run 24 with seed 826551
Run 25 with seed 370348
Run 26 with seed 717984
Run 27 with seed 273776
Run 28 with seed 395360
Run 29 with seed 746762
Run 30 with seed 219444
Kruskal-Wallis test: H-statistic = 0.7543, p-value = 6.8582e-01
```

Pseudo-code for Genetic Algorithm

1. **Function** `read_cnf(cnf_file)`
   - Open and read the CNF file.
   - Initialize `clauses` and `num_vars`.
   - Parse each line:
     a. If it specifies parameters (`p`), extract `num_vars`.
     b. If it defines a clause, convert to integers and add to `clauses`.
   - Return `clauses` and `num_vars`.
2. **Function** `check_all(clauses, state)`
   - Create a copy of `clauses`.
   - For each literal in `state`:
     a. Remove clauses satisfied by the literal.
   - Return the count of unsatisfied clauses.
3. **Function** `random_solution(num_vars)`
   - Generate a random solution by assigning each variable to `True` or `False`.
   - Return the solution.
4. **Function** `genetic_algorithm(clauses, num_vars, population_size, max_evaluations, mutation_rate)`
   - Initialize the population with random solutions.
   - Track `evaluation_count`, `best_solution`, and `best_fitness`.
   - While `evaluation_count` < `max_evaluations`:
     a. Evaluate fitness of the population.
     b. Sort population by fitness.
     c. Update `best_solution` if a better solution is found.
     d. Clone the population using `clone_population`.
     e. Generate offspring using `population_crossover`.
     f. Apply mutations to the offspring using `mutate_population`.
     g. Update population with a mix of elites and offspring.
   - Return `best_solution` and normalized fitness.

5. **Function** `clone_population(clauses, population, size)`
    - Calculate fitness for each individual in the population.
    - Use fitness-based sampling to generate a new population.
    - Return the cloned population.
6. **Function** `population_crossover(population, size)`
    - Generate new individuals by combining segments of parent solutions.
    - Return the new population.
7. **Function** `mutate_population(population, mutation_rate)`
    - Determine the number of mutations to apply.
    - Randomly select individuals and variables to mutate.
8. **Function** `run_experiment(cnf_file)`
    - Load CNF file using `read_cnf`.
    - Set algorithm parameters.
    - For each run:
        a. Set a random seed and record start time.
        b. Run `genetic_algorithm`.
        c. Track runtime and fitness.
    - Perform Kruskal-Wallis test on results.
    - Plot fitness across runs and runtime correlation.
9. **Main Execution**
    - Prompt the user to select a MAXSAT instance.
    - Map the user choice to a CNF file.
    - Run `run_experiment` with the chosen file.

Reference

Shivram, P. (2019). **Parallelization of Genetic Algorithm to Solve MAX-3SAT Problem on GPUs**. Master's thesis, University of South Florida. Available at: https://digitalcommons.usf.edu/etd/7916(Parallelization of Gene...).

Wei, Z., & Hao, J.-K. (2021). **Multistart Solution-Based Tabu Search for the Set-Union Knapsack Problem**. *Applied Soft Computing, 105*, 107260. Available at: https://hal.science/hal-03586818(MSBTS).

James, T., Rego, C., & Glover, F. (2009). **Multistart Tabu Search and Diversification Strategies for the Quadratic Assignment Problem**. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans, 39*(3), 579-596. Available at: DOI:10.1109/TSMCA.2009.2014556(Multistart_Tabu_Search_...).

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). **A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II**. *IEEE Transactions on Evolutionary Computation, 6*(2), 182-197. Available at: DOI:10.1109/TEVC.2002.996017(Deb_NSGAII).